PL2: GRAMÁTICAS Y GENERADORES AUTOMÁTICOS

PROCESADORES DEL LENGUAJE

CUBAS MARTI DAVID | 05940806K
FUENTES LOZANO ALVARO | 03225883H
TORRICO LÓPEZ DIANA | 53757599K

Tabla de contenido

1. Primera parte: generación de árboles sintácticos para lenguajes específicos 2
1.1. Árbol sintáctico de un CSV 2
1.1.1. Nivel Básico 2
1.1.2. Nivel Medio 3
1.2. Lenguaje de programación mínimo 5
1.2.1. Nivel básico 5
1.2.2. Nivel Medio 7
2. Segunda parte: Un problema de librerías (de IKEA) 11
2.1. Nivel básico 11
2.1.0. Decisiones de diseño 11
2.1.1. Analizador Léxico (Lexer) 13
2.1.2. Analizador Sintáctico (Parser) 14
2.1.2. Muebles Ikea lenguaje y árbol 15
2.2. Nivel avanzado 23
2.2.1 Enfoque general 23
2.2.2. Tabla de símbolos y estructuras de datos 23
2.2.3. Recorrido del árbol y actualización de la tabla 24
2.2.4. Salida por pantalla 24
2.2.5. Ampliaciones como consecuencia de decisiones de diseño 25
3. Anexo 1 26

1. Primera parte: generación de árboles sintácticos para lenguajes específicos

1.1. Árbol sintáctico de un CSV

1.1.1. Nivel Básico

La construcción del árbol sintético se realizó una vez implementadas las fases del analizador léxico (lexer) y el analizador sintáctico (parser) para dicho lenguaje.

Analizador léxico (LEXER):

En esta parte del código hemos definido las expresiones en las cuales se basará el analizador para identificar los diferentes tokens del lenguaje.

```
NEWLINE : [\r\n]+;      // aceptamos salto de linea en mac y windows
COMA: ',';
PUNTO_COMA: ';';
BARRA: '|';

//números
NUMERO: [0-9]+;
NUM_COMAS: "" [+\-]? NUMERO ( ',' NUMERO )? "" ;   // expresión algebraica número entre coma
TIEMPO:  "" [+\-]? NUMERO ( ',' NUMERO )? 's' "";
FECHA: NUMERO '-' NUMERO '-' NUMERO;
PAREJA_N: NUMERO '-' NUMERO;

//palabras
PALABRA: [a-zA-Z]+;
TEXTO: PALABRA (' ' PALABRA|PAREJA_P)*;  //texto de palabra separada por espacio
PAREJA_P: PALABRA ( '-' PALABRA) ;      //expresión para aceptar dos palabras con guion
```

Como podemos observar, las reglas implementadas nos dan la opción de identificar distintos tipos de números (tanto positivos, como negativos como números sin signo) además de los decimales y números que definirán segundos a los que hemos llamado TIEMPO.

De igual forma hemos contemplado distintos casos para procesar las palabras, ya pueden estar formadas por una o varias o contener un guion entre ellas.

Con estas definiciones constituimos la base del lexer, que como bien se ha mencionado antes, tratará de dividir la entrada en distintas unidades léxicas comprensibles por el parser.

Analizador Sintáctico (PARSER):

Una vez construido el analizador léxico, hemos construido la parte del código que se encarga de interpretar las distintas secuencias de tokens que han sido generadas por el analizador léxico y de organizar la estructura del lenguaje que se generará en forma del árbol sintáctico.

Si nos fijamos en el código realizado podemos observar las distintas reglas usadas para la descripción de la unión de los distintos tokens que nos proporcionaran las expresiones válidas dentro del lenguaje.

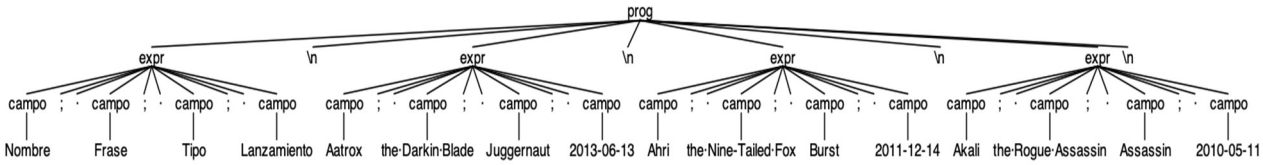
```
prog:   (expr NEWLINE)* (NEWLINE)? ;  // conjunto de lineas separadas por salto de linea


expr:   campo ((COMA | PUNTO_COMA | BARRA) ' '? campo)*   // separamos los tokenes
   ;
```

```
campo:  NUMERO
   |  NUM_COMAS
   |  TIEMPO
   |  FECHA
   |  PAREJA_N
   |  PALABRA
   |  TEXTO
   |  PAREJA_P
   |
   ;
```

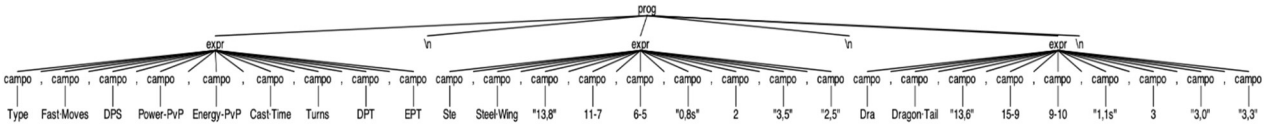
Bien se puede destacar la principal regla que es **prog** que se encarga de representar el programa completo (en este caso el csv), que es un conjunto de las expresiones (**expr**) separadas por un salto de línea.
Lo siguiente definido es una expresión (**expr**), que corresponde a una línea del archivo csv, en la que los distintos campos están separados ya bien sea por una **coma** (,), **punto y coma** (;) o una **barra** (|). Asimismo, también tuvimos en cuenta la presencia de espacios después del símbolo que los separa, para evitar errores si nos encontramos con casos como: [, texto]
Por último, tenemos la regla **campo**, donde se definen las distintas categorías que puede adoptar cada elemento dentro de una línea. En ella hemos incluido tanto los distintos tipos de número como las formas de texto definidas, a la vez que hemos contemplado la opción de tener un campo vacío (sin contenido).

ARBOL SINTÁCTICO

Una vez definidas las reglas del analizador sintáctico, con la ayuda del ANTLR hemos procedido a la construcción del árbol sintáctico. Dicho árbol representa de forma jerárquica la estructura del archivo CSV definido, mostrando cómo cada línea (**expr**) se descompone en campos (**campo**) y cómo estos, a su vez, se asocian a los distintos tipos léxicos definidos en el lexer (**números, fechas, tiempos o textos**).
CSV1:



CSV2:



1.1.2. Nivel Medio

En esta segunda parte del trabajo, desarrollamos una ampliación del nivel básico con el objetivo de obtener un árbol sintáctico legible en formato de texto plano, que mostrase únicamente los elementos indispensables del archivo CSV, evitando reproducir directamente la salida bruta generada por ANTLR.

Para ello, implementamos un listener personalizado en Java a partir de la clase Csv_gramaticaBaseListener creada automáticamente por ANTLR. Este listener, al que denominamos Pretty Listener, se encarga de recorrer el árbol sintáctico generado por el parser y construir una representación textual estructurada con indentaciones y etiquetas claras para cada nivel del árbol.

Implementación del Listener

Durante el recorrido del árbol, el *listener* va detectando las reglas gramaticales relevantes (prog, expr, campo, etc.) y las imprime en un formato jerárquico mediante el uso de un StringBuilder.
De esta forma, cada nodo del árbol se muestra en una nueva línea, precedido por un nivel de indentación proporcional a su profundidad dentro de la estructura.
El funcionamiento se basa principalmente en los métodos enter y exit de cada regla de la gramática.
Cada vez que entramos en una regla, se añade una línea con su nombre y su valor, y al salir, se reduce el nivel de indentación.
Esto nos permite representar visualmente las relaciones de dependencia entre los distintos elementos del CSV (campos, expresiones y líneas).
Por ejemplo, cuando se detecta una línea del CSV compuesta por varios campos, el *listener* imprime primero la regla expr y, debajo, los distintos campos que la forman, mostrando de manera ordenada su contenido y delimitadores.

Generación del AST textual

El código principal (MainPretty.java) se encarga de ejecutar el lexer y el parser sobre el archivo de entrada (CSV_01.txt o CSV_02.txt), recorrer el árbol con nuestro listener y guardar la salida en un archivo de texto (salida_ast.txt).
El proceso completo sigue el flujo:

```java
import java.nio.file.*;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;

public class MainPretty {
    Run | Debug
    public static void main(String[] args) throws Exception {
        if (args.length < 2) {
            System.err.println(x:"Uso: java MainPretty <entrada.txt> <salida.txt>");
            System.exit(status:1);
        }

        // Entrada
        CharStream input = CharStreams.fromPath(Paths.get(args[0]));

        // Léxico / Parser generados por ANTLR
        Csv_gramaticaLexer lexer = new Csv_gramaticaLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        Csv_gramaticaParser parser = new Csv_gramaticaParser(tokens);

        // Regla inicial
        ParseTree tree = parser.prog();

        // Recorremos con nuestro listener y escribimos fichero
        PrettyCsvListener listener = new PrettyCsvListener();
        ParseTreeWalker.DEFAULT.walk(listener, tree);

        Files.writeString(Paths.get(args[1]), listener.getText());
        System.out.println("Árbol textual generado en: " + args[1]);
    }
}
```

```
CSV
  Fila
    Palabra: Nombre
    Palabra: Frase
    Palabra: Tipo
    Palabra: Lanzamiento
  Fila
    Palabra: Aatrox
    Texto: the Darkin Blade
    Palabra: Juggernaut
    Fecha: 2013-06-13
  Fila
    Palabra: Ahri
    Texto: the Nine-Tailed Fox
    Palabra: Burst
    Fecha: 2011-12-14
  Fila
    Palabra: Akali
    Texto: the Rogue Assassin
    Palabra: Assassin
    Fecha: 2010-05-11
```

Como resultado para el primer CSV obtenemos
Este formato de salida cumple con el requisito de mostrar el árbol de manera entendible y estructurada, facilitando la interpretación del contenido del CSV sin necesidad de utilizar el visor gráfico de ANTLR.

1.2. Lenguaje de programación mínimo

1.2.1. Nivel básico

1.2.1.1. Identificación de los elementos del lenguaje E++

Para la creación del lenguaje E++, el primer paso consistió en analizar los componentes fundamentales que debían ser reconocidos de forma automática.
El objetivo fue diseñar un lenguaje con una estructura similar a la de un lenguaje de programación real, pero con una sintaxis simplificada que permitiese construir un análisis léxico y sintáctico completo.
A partir del ejemplo del enunciado pudimos identificar elementos esenciales del lenguaje. Entre ellos hemos separado por un lado a las palabras clave; como asignar, mostrar, si, no, terminar o sino. También tenemos símbolos especiales como *"???"* que quiere decir fin de condición, *"->"* que consiste en el inicio de bloque y por último *";P"* como fin de instrucción. Además de los anteriores, obviamente tenemos a los operadores aritméticos y lógicos clásicos de un lenguaje de programación. Finalmente, tenemos nombres de variables (ID), número enteros (NUMBER), reales (FLOAT), cadenas de texto (STRING) y booleanos.
Tras este análisis inicial pasamos a elaborar las reglas léxicas donde cada token identifica un símbolo del lenguaje:

```
// ====================== LÉXICO ======================

// Palabras clave
ASIGNAR  : 'asignar';
MOSTRAR  : 'mostrar';
TERMINAR : 'terminar';
SI       : 'si';
SINO     : 'sino';
NO       : 'no';

ARROW    : '->';
COND_END : '???';
SEP      : ';' 'P';

// Operadores
ASSIGN       : '=';
PLUS         : '+';
MINUS        : '-';
MUL          : '*';
DIV          : '/';

PLUS_ASSIGN  : '+=';
MINUS_ASSIGN : '-=';

AND : '&&';
OR  : '||';
NOT : '!';

EQ : '==';
NEQ: '!=';
LE : '<=';
GE : '>=';
LT : '<';
GT : '>';

// Literales
FLOAT  : [0-9]+ '.' [0-9]+;
NUMBER : [0-9]+;

TRUE   : 'verdadero';
FALSE  : 'falso';

STRING : '"' ( ~["\\] | '\\' . )* '"';

// Identificadores
ID     : [a-zA-Z_][a-zA-Z_0-9]*;

// Comentarios y espacios
COMMENT: '#' ~[\r\n]* -> skip;
WS     : [ \t\r\n]+   -> skip;
```

Además, añadimos las reglas para ignorar comentarios y espacios.
Con la parte del léxico definida, el lexer generado por ANTLR es capaz de detectar cada símbolo, palabra o literal de cualquier programa escrito en E++.
A continuación, nos tocaría construir el parser (analizador sintáctico). Una vez definidos los tokens desarrollamos el conjunto de reglas sintácticas que describen la estructura del lenguaje.
En cuanto a las estructuras principales tenemos por un lado a las asignaciones iniciales y a las reasignaciones.
A las impresiones por pantalla.

```
printStmt
    : MOSTRAR expr SEP
    ;
```

A los condicionales anidados con variantes si, sino si y sino. Permitiendo con estas reglas condicionales complejos con muchas ramas.

```
34  ifStmt
35      : condition thenHeader block (elseifHeader block)* (elseHeader block)? TERMINAR
36      ;
37
38  condition       : expr COND_END ;
39  thenHeader      : SI ARROW ;
40  elseifHeader    : SINO SI condition ARROW ;
41  elseHeader      : (NO | SINO) ARROW ;
42
```

Y las expresiones con precedencia de operadores que incluyen operaciones aritméticas, relacionales, lógicas y unarias.

```
expr        : orExpr ;
orExpr      : andExpr      ( OR  andExpr      )* ;
andExpr     : eqExpr       ( AND eqExpr       )* ;
eqExpr      : relExpr      ( (EQ|NEQ) relExpr )? ;
relExpr     : addExpr      ( (LT|LE|GT|GE) addExpr )? ;
addExpr     : mulExpr      ( (PLUS|MINUS) mulExpr )* ;
mulExpr     : unary        ( (MUL|DIV) unary )* ;
unary       : (NOT | MINUS) unary
            | atom
            ;
```

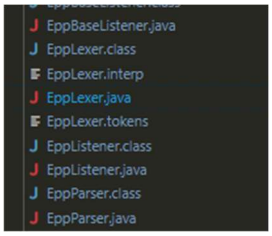
Todas estas producciones garantizan que el parser generado por ANTLR pueda reconocer la jerarquía de las operaciones y la estructura lógica del lenguaje, generando el AST correcto.

1.2.1.1. Generación del lexer y parser con ANTLR

Empleando la herramienta ANTLR, definida ya la gramática Epp.g4 (todo lo anterior en dicho archivo) mediante el comando por CMD (Windows):

```
C:\antlr\gramatica_E++>antlr Epp.g4

C:\antlr\gramatica_E++>java org.antlr.v4.Tool Epp.g4

C:\antlr\gramatica_E++>
```

De esta manera se generan automáticamente los analizadores léxico y sintáctico, siendo el lexer el que analiza los tokens, el parser el de la estructura sintáctica y el base listener el que permite recorrer el árbol generado.

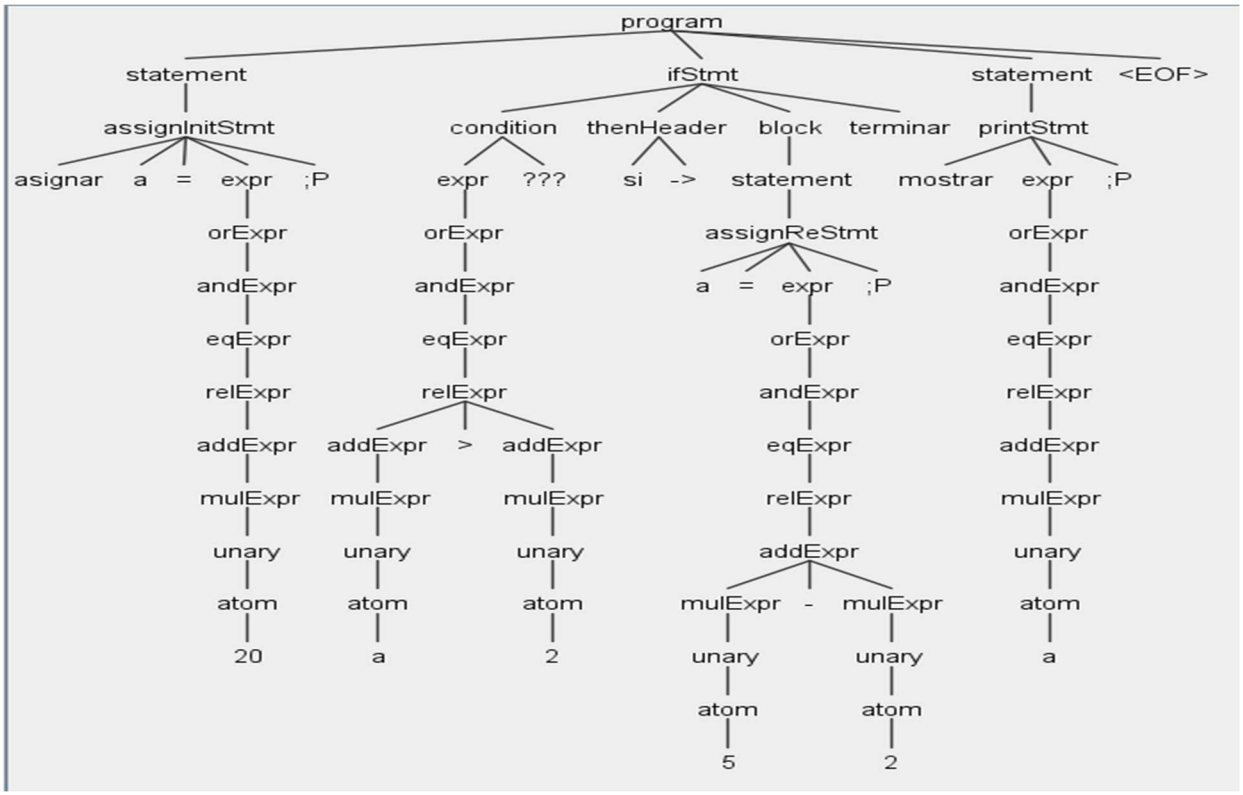


Para poder emplear los archivos java debemos compilarlos todos con una sentencia simple como **javac *.java**. Con esto ya podríamos pasar a ejecutar el analizador para un programa de ejemplo.

Teniendo como ejemplo el txt dado en el aula virtual, ejecutando las ordenes siguientes

```
C:\antlr\gramatica_E++>grun Epp program -gui

C:\antlr\gramatica_E++>java org.antlr.v4.gui.TestRig Epp program -gui
# Inicializacion de variable
asignar a = 20 ;P

# if-else sencillo
a > 2 ???
si ->
    a = 5 - 2 ;P
terminar

# imprimir por pantalla
mostrar a ;P
^Z
```

Obtenemos el árbol correspondiente generado a partir del lexer y parser.



1.2.2. Nivel Medio

1.2.2.1. Ampliaciones de la gramática

Una vez definido el núcleo del lenguaje E++ y comprobado que el analizador léxico y sintáctico eran capaces de reconocer correctamente el ejemplo básico proporcionado en el enunciado, procedimos a realizar una serie de ampliaciones controladas de la gramática con el objetivo de aproximar E++ a un lenguaje de programación más habitual.
Introdujimos cuatro extensiones principales, todas ellas reflejadas en el fichero Epp.g4 mostrado anteriormente:

1. Operadores aritméticos completos y unarios
2. Soporte de booleanos y operadores lógicos
3. Estructura condicional con sino si (else-if)
4. Asignaciones compuestas += y -=

Operadores aritméticos completos y unarios

En la especificación original solo aparecía la suma y la resta en contextos muy sencillos. Para asemejar E++ a un lenguaje real, decidimos añadir los operadores de multiplicación (*) y división (/) y la negación del tipo “-x”.

En el léxico esto se ve reflejado en los tokens:

```
87   PLUS        : '+';
88   MINUS       : '-';
89   MUL         : '*';
90   DIV         : '/';
```

Y en el parser organizamos la precedencia de operadores en varios niveles:

```
addExpr     : mulExpr      ( (PLUS|MINUS) mulExpr )* ;
mulExpr     : unary        ( (MUL|DIV) unary )* ;
unary       : (NOT | MINUS) unary
            | atom
            ;
```

De este modo el árbol sintáctico respeta la precedencia estándar de multiplicación y división por encima de la suma y la resta, y el operador unario antes que los binarios permitiendo expresiones como “ z = -x * 2 + 3 ;P con un comportamiento similar al de los lenguajes de programación habituales.

Booleanos y operadores lógicos

Aunque el lenguaje básico ya contemplaba las comparaciones relacionales habituales (<, >, <=, >=, ==, !=), ampliamos el conjunto de operadores para incluir también conectores lógicos (&&, ||, !) y valores booleanos (verdadero, falso), lo que permite expresar condiciones compuestas y dotar al lenguaje de una semántica más cercana a los lenguajes imperativos convencionales.

```
AND : '&&';
OR  : '||';
NOT : '!';
```

```
TRUE  : 'verdadero';
FALSE : 'falso';
```

Ampliamos la jerarquía de expresiones para poder incluirlos de manera adecuada.

```
expr        : orExpr ;
orExpr      : andExpr      ( OR  andExpr      )* ;
andExpr     : eqExpr       ( AND eqExpr       )* ;
```

Condicionales con sino (else-if)

El enunciado solo habla de condicionales normales y anidados, así que decidimos incorporar el tipo de condicional else-if modificando el ifStmt

```
ifStmt
    : condition thenHeader block (elseifHeader block)* (elseHeader block)? TERMINAR
    ;
```

```
elseifHeader   : SINO SI condition ARROW ;
```

Asignaciones compuestas += y -=

Al igual que en los lenguajes de programación comunes, en lugar de poner sentencias como x = x + 1 ;P, decidimos incorporar las asignaciones compuestas para hacerlo más parecido a un lenguaje común.

```
PLUS_ASSIGN  : '+=';
MINUS_ASSIGN : '-=';
```

En el léxico añadimos sus correspondientes tokens.

```
assignReStmt
    : ID (ASSIGN | PLUS_ASSIGN | MINUS_ASSIGN) expr SEP
    ;
```

Y en el parser de la siguiente manera. Pudiendo así utilizar expresiones como x += 1 ;P

1.2.2.2. Generación de un AST legible en texto plano

Tras la generación correcta del árbol sintáctico mediante ANTLR, se nos pedía mostrar dicho árbol en un formato legible y simplificado, que contuviera únicamente los elementos esenciales del programa, evitando replicar directamente la salida visual de ANTLR o su estructura interna completa.

Para cumplir este requisito, desarrollamos un listener personalizado a partir de la clase EppBaseListener generada automáticamente por ANTLR. Este componente, denominado AstPrinterListener, recorre el árbol sintáctico y construye una representación textual jerarquizada del mismo, almacenándola en un StringBuilder.

Durante la ejecución del recorrido, vamos indentando los niveles de anidamiento mediante espacios para reflejar de forma clara la estructura jerárquica de las instrucciones. Cada nodo relevante del árbol (asignaciones, condiciones, bloques, expresiones o literales) se imprime en una línea, omitiendo los detalles sintácticos irrelevantes como paréntesis o separadores ;P.

El proceso general se basa en la siguiente estructura:

1. Recorrer el árbol utilizando el método ParseTreeWalker.DEFAULT.walk(listener, tree); tras haber obtenido el árbol tree a partir del parser (parser.program()).
2. Detectar cada tipo de nodo mediante los métodos enter y exit del listener, generando una línea de texto descriptiva para cada regla gramatical relevante (por ejemplo, AssignInit, If, Cond, Then, AddSub, Rel op=>, Int 5, etc.).
3. Controlar la indentación incrementando o decrementando el nivel de tabulación según se entra o sale de bloques o expresiones anidadas.
4. Guardar la salida en un archivo .txt con un formato limpio y comprensible.

El código del programa principal, denominado MainAstListener, ejecuta automáticamente este proceso con el siguiente flujo:

```
5   public class MainAstListener {
        Run | Debug
6       public static void main(String[] args) throws Exception {
7           if (args.length < 2) {
8               System.err.println(x:"Uso: java MainAstListener <entrada.epp> <salida.txt>");
9               return;
10          }
11          CharStream input = CharStreams.fromPath(Paths.get(args[0]));
12          EppLexer lexer = new EppLexer(input);
13          CommonTokenStream tokens = new CommonTokenStream(lexer);
14          EppParser parser = new EppParser(tokens);
15
16          ParseTree tree = parser.program();
17
18          AstPrinterListener listener = new AstPrinterListener();
19          ParseTreeWalker.DEFAULT.walk(listener, tree);
20
21          Files.writeString(Paths.get(args[1]), listener.getText());
22          System.out.println("AST guardado en: " + Paths.get(args[1]).toAbsolutePath());
23      }
24  }
```

De esta forma su modo de uso es analizar un txt y generar uno nuevo de salida que contiene el árbol en formato de texto plano.

Por ejemplo, para el siguiente txt de prueba:

Obteniendo el txt de la derecha, el cual muestra la jerarquía en forma de árbol listado. Esta salida es concisa, clara y fiel a la estructura lógica del programa, mostrando únicamente los elementos necesarios para entender la jerarquía de sentencias y expresiones.

```
# --- Inicializaciones ---
asignar x = 3 ;P
asignar y = 4.5 ;P
asignar ok = falso ;P
asignar z = 0 ;P

# --- Unario y lógicos ---
ok = !ok ;P          # ahora true
z = -x ;P            # unario negativo

# --- Aritmética + compuestas ---
x += 2 * 5 ;P        # usa * y +=
y -= 1.5 ;P          # usa -=

# --- If con OR, AND, relacionales y 'sino si' ---
(ok && x > 10) || (y >= 5) ???
si ->
  mostrar "rama then" ;P
sino si x == 13 ???
->  mostrar "elseif x == 13" ;P
sino ->
  mostrar "rama else" ;P
terminar

# --- Salidas ---
mostrar x ;P
mostrar y ;P
mostrar z ;P
```

```
Program
  AssignInit id=x
    Int 3
  AssignInit id=y
    Float 4.5
  AssignInit id=ok
    Bool falso
  AssignInit id=z
    Int 0
  Assign id=ok op==
    Unary op=!
      Id ok
  Assign id=z op==
    Unary op=u-
      Id x
  Assign id=x op=+=
    MulDiv
      Int 2
      Int 5
  Assign id=y op=-=
    Float 1.5
  If
    Cond
      Or
        And
          Id ok
          Rel op=>
            Id x
            Int 10
        Rel op=>=
          Id y
          Int 5
    Then
      Print
        String "rama then"
    Cond
      Eq op===
        Id x
        Int 13
    ElseIf#1
      Print
        String "elseif x == 13"
    Else
      Print
        String "rama else"
  Print
    Id x
  Print
    Id y
  Print
    Id z
```

Gracias a esta implementación, logramos un AST legible y exportable, cumpliendo con el segundo requisito del nivel medio, y facilitando tanto la depuración del lenguaje como su posible ampliación futura hacia fases de interpretación o generación de código.

2. Segunda parte: Un problema de librerías (de IKEA)

2.1. Nivel básico

A la hora de definir un lenguaje de programación que fuese capaz de simular el manual de instrucciones para la construcción de un mueble de IKEA, el primer paso fue realizar un análisis de distintos manuales de montaje proporcionados. Nuestro principal objetivo era extraer los elementos comunes que aparecen en todos ellos: herrajes necesarios, herramientas utilizadas, verbos de acción y pasos de montaje… Dicho análisis permitió establecer una base común de conceptos y acciones, que nos a servido como estructura del lenguaje diseñado. Para esta primera parte es cierto que contamos con la ayuda de chatgpt 5 pero, nos dimos cuenta de que no funcionaba bien a la hora de procesar las imágenes de los manuales, aunque nos sirvo para tener una primera idea, de cómo ir creando nuestro lenguaje.

Una vez definidos los elementos del dominio (piezas, herramientas, acciones y cantidades) nos centramos en diseñar la gramática del lenguaje, que describe las posibles combinaciones entre dichos componentes. Esta gramática fue la base para implementar el analizador léxico y el analizador sintáctico. Sabiendo que el lexer se encarga transformar el texto fuente en una secuencia de tokens, mientras que el parser comprueba que dicha secuencia respete las reglas gramaticales y genera la estructura jerárquica correspondiente. Es cierto que sí que hemos usamos el código que nos proporcionó el chat para cada una de las partes del lexer y el parser, pero al igual que en la primera parte nos encontramos con varios errores que fuimos arreglando y mejorando a lo largo del trabajo.

El objetivo de esta parte del trabajo consistió en crear un lenguaje formal, no ambiguo capaz de representar las instrucciones de montaje para cualquier mueble de IKEA. Con ello pretendíamos demostrar como los principios de construcción de compiladores se pueden aplicar a la definición de lenguajes específicos de dominio (DSL), capaces de traducir procedimientos cotidianos en estructuras formales comprensibles para una máquina.

2.1.0. Decisiones de diseño

A partir de esta definición inicial del lenguaje y de su gramática básica, se fueron tomando una serie de decisiones de diseño que permitieron refinar, ampliar y adaptar el lenguaje a los distintos manuales analizados. Estas decisiones, han ido surgiendo durante el proceso de desarrollo de la traducción de los distintos manuales al lenguaje formal, se detallan a continuación.

Una de las primeras decisiones fue incluir en la regla **piezaRef** la posibilidad de hacer referencia tanto a piezas como al propio mueble. Esta ampliación se introdujo porque observamos que en algunos manuales se hacía referencia al mueble completo, incluso cuando aún no estaba totalmente montado. Con esta decisión, el lenguaje permite reconocer el mueble como una pieza más dentro del proceso de ensamblaje, reflejando así con mayor precisión la lógica de los pasos reales. A su vez también incluimos la opción de identificar las piezas sin la necesidad de usar “” por eso incluimos el token **ID.**

Como decisión de diseño explícita, optamos por omitir todas aquellas acciones relacionadas únicamente con cambiar la orientación del mueble —por ejemplo, voltearlo, inclinarlo o colocarlo temporalmente boca abajo—. Consideramos que estos movimientos forman parte del proceso físico natural del montaje y que los manuales de IKEA los

representan de manera visual más que como pasos "informativos". Incluirlos en el lenguaje habría añadido ruido y complejidad sin aportar información relevante para el análisis sintáctico ni para la descripción estructural del montaje. Por ello, decidimos tratar estos cambios de orientación como **acciones implícitas** que el montador asume de manera intuitiva, centrándonos en modelar únicamente las operaciones que afectan directamente a la estructura, conexión y fijación de piezas y herrajes.
En cuanto a la estructura del programa, decidimos que al inicio se definieran las piezas, herrajes y herramientas utilizadas durante el montaje. Estas definiciones iniciales facilitan la legibilidad y el control de los elementos disponibles. Sin embargo, durante el desarrollo comprobamos que era posible escribir manuales completos sin necesidad de definir previamente todos los componentes, simplemente mencionándolos en los pasos de montaje. Por ello, en la práctica hemos mantenido ambos enfoques: uno más formal, con las definiciones al principio, y otro más directo, que las omite. Esta decisión se refleja, por ejemplo, en el montaje del mueble *Billy*, donde optamos por no incluir las definiciones iniciales. Esta decisión de diseño no produce errores como tal, pero es cierto que ha sido una decisión tomada para una mejor visualización de nuestro lenguaje de programación y por ello en estas definiciones se incluyen las palabras clave entre "".
Otra decisión importante fue la inclusión de **mecanismos de repetición**, que aparecieron de forma natural al observar que en los manuales muchas acciones se repiten sobre piezas similares. Por un lado, incorporamos la repetición dentro de una acción (por ejemplo, repetir 2) para indicar que una misma operación debe realizarse varias veces sobre elementos equivalentes. Por otro lado, creamos la posibilidad de repetir pasos completos, de modo que un conjunto de instrucciones pueda aplicarse a distintas piezas o partes del mueble.
Posteriormente añadimos también la opción de **deshacer paso**, inspirada en el manual del mueble *Smagora*, donde ciertas operaciones se ejecutan, luego se revierten y se repiten más adelante. En lugar de crear un nuevo verbo ("sacar"), hemos optado por incluir la posibilidad de deshacer, por coherencia semántica y simplicidad gramatical.
En el ámbito léxico, inicialmente decidimos ignorar las tildes y caracteres especiales del español para evitar complicaciones técnicas. No obstante, tras debatirlo, se ha considerado que nuestro lenguaje debía mantener la fidelidad al idioma en el que se basa, por lo que incluimos las vocales acentuadas y la letra "ñ" dentro de los identificadores válidos.

De igual forma, hemos optado por no permitir el uso de mayúsculas ni en los nombres de piezas ni en los pasos, con el objetivo de mantener un estilo uniforme y evitar ambigüedades en el análisis.
Finalmente, en las últimas etapas de diseño hemos introducido dos ampliaciones que mejoraron considerablemente la expresividad del lenguaje:

- La posibilidad de utilizar identificadores con múltiples palabras unidas por guiones bajos, como balda_grande_lateral, para reflejar los nombres largos habituales en los manuales.
- La inclusión de varias acciones dentro de un mismo paso, mediante el uso de comas y la conjunción *y*. Esta decisión permite representar con fidelidad aquellos pasos del manual que agrupan varias operaciones paralelas, como "colocar los herrajes delanteros y traseros", manteniendo una estructura sintáctica clara y natural.

En conjunto, todas estas decisiones nos han permitido construir una gramática que no solo reproduce la lógica secuencial de los manuales de IKEA, sino que también ofrece una

sintaxis intuitiva y cercana al lenguaje humano, facilitando tanto la lectura como la traducción automática de instrucciones de montaje.

2.1.1. Analizador Léxico (Lexer)

Una vez establecidas las principales decisiones de diseño y la estructura general del lenguaje, se procedió a implementar el analizador léxico, encargado de identificar y clasificar los elementos básicos del programa.

En primer lugar, se definió un conjunto limitado de palabras clave y estructuras gramaticales, que constituyen el vocabulario básico del leguaje. En este conjunto hemos incluido tanto los **verbos de acción** *(usar, atornillar, colgar…)* como los **sustantivos** a los que asociamos los objetos (*pieza, herramienta, herraje…*) y los **conectores** que hemos visto necesarios para describir las relaciones entre ellos (*con, en, usando, y …*).

De esta forma, se ha delimitado el alfabeto y los símbolos terminales del lenguaje, para que el analizador léxico pueda reconocer y clasificar cada elemento de la entrada.

A continuación, se puede ver la definición formal de los principales tokens de este lenguaje, expresados mediante expresiones regulares, las cuales se encargan de definir el patrón de los lexemas válidos para cada categoría.

```
// LEXER
MUEBLE: 'mueble';
HERRAMIENTA: 'herramienta';
HERRAJE: 'herraje';
PIEZA: 'pieza';
CANTIDAD: 'cantidad';
PASO: 'paso';


//verbos
USAR: 'usar';
UNIR: 'unir';
FIJAR: 'fijar';
INSERTAR: 'insertar';
REPETIR: 'repetir';
DESHACER: 'deshacer';
COMPROBAR: 'comprobar';
ATORNILLAR: 'atornillar';
MARTILLAR: 'martillar';
COLGAR: 'colgar';
MARCAR: 'marcar';
PEGAR: 'pegar';
COLOCAR: 'colocar';
NIVELAR: 'nivelar';


FIN: 'fin';
CON: 'con';
EN: 'en';
A: 'a';
Y: 'y';
USANDO: 'usando';
PARED: 'pared';


NUM: [0-9]+;
```

```
CADENA: "" (~["\r\n])* "";
ID: [a-zA-ZáéíóúÁÉÍÓÚñÑ_] [a-zA-Z0-9_áéíóúÁÉÍÓÚñÑ]*;
COMA: ',';
PUNTOYCOMA: ';';
DOS_PUNTOS: ':';
ESPACIO: [ \t\r\n]+ -> skip;
```

2.1.2. Analizador Sintáctico (Parser)

Una vez hemos definido los tokens básicos mediante el analizador léxico, pasamos a implementar el analizador sintáctico. Responsable que se encarga de definir la estructura jerárquica del lenguaje, es decir, definir como se combinan los tokens producidos por el lexer para la formación de construcciones válidas. La función principal del parser es verificar que la secuencia de tokens sigue las reglas gramaticales del lenguaje y en función generar el correspondiente árbol sintáctico, que representa la organización lógica de las instrucciones.

En este caso, hemos definido el nodo principal del árbol como el programa, que agrupa los tres bloques fundamentales en los que hemos dividido el lenguaje: **definición del mueble,** las **definiciones auxiliares** (herramientas, piezas, herrajes) y por último los **pasos de montaje** (donde se refleja paso a paso la secuencia de acciones a realizar para montar dicho mueble).

Cada producción de la gramática genera un subárbol independiente. Por ejemplo, el pasoMontaje constituye una rama con su verbo de acción, los objetos implicados y los modificadores adicionales. A la hora de realizar el lenguaje tuvimos en cuenta varias decisiones de diseño que vimos oportunas, como dichos modificadores adicionales para hacer el lenguaje más simple como añadir los pasos de repetir y la cantidad de veces que repetimos ese mismo paso. A su vez también contamos con reglas auxiliares como herrajeLista que nos ayuda a manejar listas concatenadas, mediante comas o conjunciones para poder indicar que en un mismo paso que podemos usar más de un herraje, o piezaRef donde hemos incluido mueble y cadena para distinguir entre referencias a elementos concretos o cuando tenemos el mueble completo o nuevos elementos como en smagora que añade un mueble nuevo.

En el siguiente bloque de texto se puede ver la versión final de la gramática implementada:

```
grammar mueble_gramatica;

// PARSER
programa
   : definicionMueble seccionDefiniciones instruccion FIN PUNTOYCOMA?
   ;

definicionMueble
   : MUEBLE CADENA PUNTOYCOMA
   ;

// definiciones que no son mueble
seccionDefiniciones
   : (usoHerramienta
    | definicionPieza
    | definicionHerraje)*
   ;


definicionHerraje
   : HERRAJE CADENA CANTIDAD NUM PUNTOYCOMA
   ;
```

```
usoHerramienta
   : USAR HERRAMIENTA CADENA PUNTOYCOMA
   ;

definicionPieza
   : PIEZA CADENA CANTIDAD NUM PUNTOYCOMA
   ;

nombre: CADENA | ID;

// pasos del montaje
instruccion
   : (pasoMontaje)+
   ;

pasoMontaje
   : PASO NUM DOS_PUNTOS accionLista (REPETIR NUM)? PUNTOYCOMA
   ;

accionLista
   : accion (COMA accion)* (Y accion)?
   ;
accion:
   // Operaciones de unión/fijación
   (UNIR | FIJAR) (piezaRef|herrajeItem) ((CON | A | EN) piezaRef)? (USANDO herrajeLista (CANTIDAD NUM)?)?

   // Pared y mueble
   | ATORNILLAR MUEBLE EN PARED USANDO herrajeLista

   // Colgar
   | COLGAR piezaRef EN PARED USANDO herrajeLista (CANTIDAD NUM)?

   // Atornillar/insertar/martillar herrajes o piezas
   | (ATORNILLAR | MARTILLAR | INSERTAR | PEGAR | COLOCAR) (piezaRef | herrajeItem) (A | EN) (piezaRef |
PARED | herrajeItem) (USANDO herrajeLista (CANTIDAD NUM)?)?

   | MARCAR (piezaRef | herrajeItem) (EN (piezaRef|PARED))? USANDO herramientaRef
   | NIVELAR piezaRef USANDO herramientaRef

   //repetir paso
   | REPETIR PASO NUM (NUM)?
   | DESHACER PASO NUM
   ;

// referencias
piezaRef       : nombre | MUEBLE ;

herrajeRef     : nombre | NUM ;
herrajeItem    : herrajeRef (CANTIDAD NUM)? ;
herrajeLista   : herrajeItem (COMA herrajeItem)* (Y herrajeItem)? ;

herramientaRef  : nombre ;
```

2.1.2. Muebles Ikea lenguaje y árbol

El árbol sintáctico resultante generado por el *parser* permite visualizar la estructura lógica de cada montaje:

- El nodo raíz **programa** contiene los subnodos **definicionMueble, seccionDefiniciones e instrucción.**
- Cada **pasoMontaje** se descompone en un verbo principal y sus argumentos asociados.
- Las hojas del árbol corresponden a los terminales (tokens) reconocidos por el *lexer*.

De esta forma facilitamos la comprensión del lenguaje, además de su posible ampliación futura. Por ejemplo, podría adaptarse para generar automáticamente instrucciones paso a paso o para verificar la coherencia entre las piezas y herrajes definidos.
En la siguiente sección podemos ver los diferentes lenguajes generados para cada mueble proporcionado. A través de estos ejemplos se puede observar cómo las reglas de la gramática permiten describir operaciones de distinta complejidad, manteniendo una estructura uniforme en todos los casos.
Dado que los árboles sintácticos completos generados para cada mueble son demasiado extensos y presentan problemas de visualización en formato Word, los hemos incluido en el **Anexo I** de esta memoria. En dicho anexo pueden consultarse los árboles correspondientes a cada fichero `.txt`, junto con representaciones parciales que muestran la jerarquía interna de los nodos principales.

BAGGEBO

Traducción del manual de instrucciones:

```
mueble "BAGGEBO";

usar herramienta.    "destornillador";

pieza "panel_medio"        cantidad 4;
pieza "barra_largo"        cantidad 4;
pieza "panel_trasero"      cantidad 1;

herraje "10119123" cantidad 38;
herraje "10003068" cantidad 4;
herraje "100827"   cantidad 2;
herraje "10062215" cantidad 2;
herraje "tornillo" cantidad 2;

paso 1: atornillar panel_medio a barra_largo usando 10119123 cantidad 2 repetir 2;
paso 2: repetir paso 1 3;
paso 3: atornillar barra_largo a panel_medio usando 10119123 cantidad 8 repetir 1;
paso 4: atornillar panel_trasero a panel_medio usando 10119123 cantidad 4;
paso 5: atornillar 10003068 a barra_largo usando 10003068 cantidad 4;
paso 6: atornillar 10062215 a panel_medio usando 100827 cantidad 1 y 10119123 cantidad 1 repetir 1;
paso 7: atornillar mueble en pared usando tornillo cantidad 2;

fin;
```

BILLY

Traducción del manual de instrucciones:

```
mueble "BILLY";



paso 1: insertar 101351 cantidad 16 en balda;
paso 2: atornillar 118331 cantidad 12 en balda_lateral;
paso 3: fijar balda a balda_lateral;
paso 4: atornillar 119081 cantidad 4 a balda;
paso 5: atornillar 119081 cantidad 2 a balda;
paso 6: fijar balda_lateral a balda;
paso 7: atornillar balda_lateral a balda usando 119081 cantidad 6;
paso 8: marcar balda usando lapiz;
paso 9: colocar fondo en mueble;
paso 10: marcar fondo;
paso 11: atornillar fondo a mueble usando 101201 cantidad 1;
paso 12: fijar mueble usando 100823 cantidad 2, 106989 cantidad 2, 109041 cantidad 2, 100823 cantidad 2 y 106989;
paso 13: colocar 131372 cantidad 16 en balda_lateral;
paso 14: colocar balda en mueble;

fin;
```

FLYSTA

Traducción del manual de instrucciones:

```
mueble "FLYSTA";

usar herramienta "martillo";

pieza "panel_centro_horizontal"        cantidad 6;
pieza "panel_centro_vertical"          cantidad 4;
pieza "panel_lateral"                  cantidad 2;

herraje "115753" cantidad 2;
herraje "115754" cantidad 2;
herraje "104321" cantidad 8;
herraje "101339" cantidad 6;
herraje "103693" cantidad 2;
herraje "100372" cantidad 4;
herraje "100092" cantidad 1;
herraje "121030" cantidad 1;
herraje "153550" cantidad 2;
herraje "101345" cantidad 16;

paso 1:  atornillar panel_centro_horizontal a panel_lateral usando 100092 cantidad 1, 121030 cantidad 1 y 104321
cantidad 2;
paso 2:  martillar panel_centro_vertical a panel_centro_horizontal usando 101345 cantidad 2;
paso 3:  martillar panel_centro_horizontal a panel_lateral usando 101345 cantidad 2;
paso 4:  martillar 101339 cantidad 2 en panel_centro_horizontal;
paso 5:  unir panel_centro_vertical a panel_centro_horizontal;
paso 6:  martillar panel_centro_horizontal a panel_lateral usando 101345 cantidad 2;
paso 7:  martillar 101339 cantidad 2 en panel_centro_horizontal;
paso 8:  unir panel_centro_vertical a panel_centro_horizontal;
paso 9:  martillar panel_centro_horizontal a panel_lateral usando 101345 cantidad 2;
paso 10: martillar 101339 cantidad 2 en panel_centro_horizontal;
paso 11: unir panel_centro_vertical a panel_centro_horizontal;
paso 12: martillar 101345 cantidad 8 en medioMontado;
paso 13: unir panel_lateral a medioMontado;
paso 14: atornillar panel_centro_horizontal a panel_lateral usando 100092 cantidad 1, 121030 cantidad 1 y 104321
cantidad 6;
paso 15: pegar 153550 a mueble;
paso 16: atornillar 115753 a mueble usando 100372 cantidad 2 repetir 1;
paso 17: marcar 115753 en pared usando lapiz;
paso 18: atornillar 115753 a pared usando tornillo repetir 1;
paso 19: atornillar 103693 a 115753 usando destornillador repetir 1;
paso 20: colocar 115754 en 115753;

fin;
```

HEMNES

Traducción del manual de instrucciones:

```
mueble "Hemnes";

pieza "balda_grande_lateral" cantidad 2;
pieza "separador"            cantidad 1;
pieza "enganche_extremo"     cantidad 4;
pieza "barra_lateral_larga"  cantidad 4;
pieza "panel_medio"          cantidad 5;
pieza "panel_fondo_pequeño"  cantidad 2;
pieza "panel_base"           cantidad 1;
pieza "panel_tapa"           cantidad 1;

usar herramienta "destronillador punta plana";
```

```
usar herramienta "destronillador estrella";
usar herramienta "lapiz";
usar herramienta "nivelador";

herraje "104171" cantidad 12;
herraje "101350" cantidad 16;
herraje "118331" cantidad 12;
herraje "113434" cantidad 12;
herraje "102728" cantidad 4;
herraje "100211" cantidad 13;
herraje "122723" cantidad 9;
herraje "106414" cantidad 8;
herraje "119256" cantidad 2;
herraje "100823" cantidad 4;
herraje "109560" cantidad 11;

paso 1:  atornillar 118331 cantidad 4 en balda_grande_lateral usando destronillador repetir 2;
paso 2:  unir 101350 cantidad 3 con separador repetir 2;
paso 3:  atornillar 100211 en balda_grande_lateral usando 100001 repetir 2;
paso 4:  atornillar enganche_extremo en balda_grande_lateral usando 100211 cantidad 2 y 100001 repetir 4;
paso 5:  unir enganche_extremo usando 102728 cantidad 4;
paso 6:  atornillar barra_lateral_largo a balda_grande_lateral usando 101350;
paso 7:  repetir paso 6;
paso 8:  repetir paso 6 2;
paso 9:  atornillar 113434 cantidad 1 a barra_lateral_largo repetir 4;
paso 10: atornillar 118331 cantidad 2 a separador repetir 1;
paso 11: atornillar panel_medio a separador usando 122332 cantidad 2;
paso 12: fijar barra_lateral_largo a balda_grande_lateral usando 101350 repetir 4;
paso 13: atornillar 113434 cantidad 2 a barra_lateral_largo repetir 2;
paso 14: fijar panel_tapa a mueble usando 101350 cantidad 2;
paso 15: atornillar panel_medio a mueble usando 100211 cantidad 7 y 100001;
paso 16: colocar panel_fondo_pequeño a mueble;
paso 17: repetir paso 16;
paso 18: repetir paso 16;
paso 19: repetir paso 16;
paso 20: atornillar panel_fondo_pequeño a mueble usando 109560 cantidad 3 y 122723 cantidad 3;
paso 21: repetir paso 20 2;
paso 22: nivelar mueble usando nivelador;
paso 23: atornillar mueble a pared usando 109560 cantidad 2, 100823 cantidad 4, tornillo cantidad 2 y 119256 cantidad
2;
paso 24: insertar 104171 cantidad 4 en barra_lateral_larga;
paso 25: colocar panel_base en mueble;
paso 26: insertar 104171 cantidad 8 en barra_lateral_larga_trasera y colocar 106414 cantidad 8 en
barra_lateral_larga_delantera;
paso 27: colocar panel_medio en mueble repetir 4;

fin;
```

KALLAX

Traducción del manual de instrucciones: este manual tenía dos diferentes versiones

1

```
mueble "Kallax";

usar herramienta "destronillador punta plana";
usar herramienta "destronillador estrella";
usar herramienta "lapiz";
usar herramienta "martillo";
usar herramienta "nivel";

pieza "panel_centro_horizontal"        cantidad 6;
pieza "panel_centro_vertical"          cantidad 4;
pieza "balda_grande"                   cantidad 2;
```

```
pieza "balda_mediana"                cantidad 2;


herraje "104321" cantidad 8;
herraje "101339" cantidad 22;
herraje "115753" cantidad 2;
herraje "115754" cantidad 2;
herraje "103603" cantidad 2;
herraje "100372" cantidad 4;
herraje "100092" cantidad 1;
herraje "121030" cantidad 1;
herraje "10002300" cantidad 2;


paso 1:  atornillar balda_mediana en balda_grande usando 104321 cantidad 2, 121030 y 100092;
paso 2:  martillar panel_centro_vertical a balda_mediana usando 101339 cantidad 2;
paso 3:  martillar panel_centro_horizontal a balda_grande usando 101339 cantidad 2;
paso 4:  martillar panel_centro_horizontal a panel_centro_vertical usando 101339 cantidad 2;
paso 5:  fijar panel_centro_vertical a panel_centro_horizontal;
paso 6:  repetir paso 3;
paso 7:  repetir paso 4;
paso 8:  repetir paso 5;
paso 9:  repetir paso 3;
paso 10: repetir paso 4;
paso 11: repetir paso 5;
paso 12: martillar 101339 cantidad 8 en medioMontado;
paso 13: unir balda_grande a medioMontado;
paso 14: colocar balda_mediana a medioMontado y atornillar balda_mediana a balda_grande usando 104321 cantidad
6, 100092 y 121030;
paso 15: pegar 121030 cantidad 4 en balda_mediana;
paso 16: atornillar 115753 a mueble usando 100372 cantidad 2 repetir 2;
paso 17: marcar 115753 cantidad 2 en pared usando lapiz;
paso 18: atornillar tornillo cantidad 2 en pared;
paso 19: colocar mueble en pared usando 103693 cantidad 2;

fin;
```

2

```
mueble "Kallax";

usar herramienta "destronillador punta plana";
usar herramienta "destronillador estrella";
usar herramienta "lapiz";
usar herramienta "martillo";
usar herramienta "nivel";

pieza "panel_centro_horizontal"        cantidad 6;
pieza "panel_centro_vertical"          cantidad 4;
pieza "balda_grande"                cantidad 2;
pieza "balda_mediana"                cantidad 2;


herraje "104321" cantidad 8;
herraje "101339" cantidad 22;
herraje "115753" cantidad 2;
herraje "115754" cantidad 2;
herraje "103603" cantidad 2;
herraje "100372" cantidad 4;
herraje "100092" cantidad 1;
herraje "121030" cantidad 1;
herraje "10002300" cantidad 2;
```

```
paso 1:  atornillar balda_mediana en balda_grande usando 104321 cantidad 2, 121030 y 100092;
paso 2:  martillar panel_centro_vertical a balda_mediana usando 101339 cantidad 2;
paso 3:  martillar panel_centro_horizontal a balda_grande usando 101339 cantidad 2;
paso 4:  martillar panel_centro_horizontal a panel_centro_vertical usando 101339 cantidad 2;
paso 5:  fijar panel_centro_vertical a panel_centro_horizontal;
paso 6:  repetir paso 3;
paso 7:  repetir paso 4;
paso 8:  repetir paso 5;
paso 9:  repetir paso 3;
paso 10: repetir paso 4;
paso 11: repetir paso 5;
paso 12: martillar 101339 cantidad 8 en medioMontado;
paso 13: unir balda_grande a medioMontado;
paso 14: colocar balda_mediana a medioMontado y atornillar balda_mediana a balda_grande usando 104321 cantidad
6, 100092 y 121030;
paso 15: pegar 121030 cantidad 8 en balda_grande;
paso 16: atornillar 115753 a balda_grande usando 100372 cantidad 2 repetir 2;
paso 17: marcar 115753 cantidad 2 en pared usando lapiz;
paso 18: atornillar tornillo cantidad 2 en pared;
paso 19: colocar mueble en pared usando 103693 cantidad 2;

fin;
```

LACK

Traducción del manual de instrucciones: este manual contaba también con 2 versiones

1

```
mueble "LACK";

pieza "balda_pequeña"      cantidad 4;
pieza "balda_grande"       cantidad 4;


herraje "104323" cantidad 14;
herraje "109053" cantidad 9;
herraje "100001" cantidad 1;
herraje "120944" cantidad 3;

paso 1: atornillar balda_pequeña en balda_grande usando "104323" cantidad 2, 100001 repetir 6;
paso 2: atornillar 120944 en balda_grande usando 104323 cantidad 2;
paso 3: marcar 120944 en pared usando lapiz;
paso 4: atornillar "LACK" a pared usando "109053" cantidad 2;

fin;
```

2

```
mueble "LACK";

pieza "balda_pequeña"      cantidad 4;
pieza "balda_grande"       cantidad 4;

herraje "104323" cantidad 14;
herraje "109053" cantidad 9;
herraje "100001" cantidad 1;
herraje "120944" cantidad 3;

paso 1: atornillar balda_pequeña en balda_grande usando "104323" cantidad 2, 100001 repetir 6;
paso 2: atornillar 120944 en balda_grande usando 109053 cantidad 3 repetir 3;
paso 3: atornillar mueble a pared usando tornillo;
paso 4: repetir paso 3 y nivelar mueble usando nivelador;
paso 5: repetir paso 3;

fin;
```

VESKEN

Traducción del manual de instrucciones:

```
mueble "VESKEN";

pieza "bandeja"              cantidad 4;
pieza "barra_lateral"        cantidad 12;
pieza "barra_suelo"          cantidad 4;
pieza "enganche_bandeja"     cantidad 4;
pieza "enganche_pared"       cantidad 2;

usar herramienta "martillo";
usar herramienta "lapiz";
usar herramienta "martillo";

herraje "tornillo" cantidad 2;

paso 1:  unir barra_suelo a bandeja repetir 4;
paso 2:  unir barra_lateral a bandeja repetir 4;
paso 3:  unir bandeja a barra_lateral repetir 4;
paso 4:  unir barra_lateral a bandeja repetir 4;
paso 5:  unir bandeja a barra_lateral repetir 4;
paso 6:  unir barra_lateral a bandeja repetir 4;
paso 7: unir bandeja a barra_lateral repetir 4;
paso 8: unir enganche_bandeja a bandeja repetir 4;
paso 9: marcar bandeja usando lapiz;
paso 10: marcar enganche_pared en pared usando lapiz repetir 2;
paso 11: atornillar enganche_pared en pared usando tornillo repetir 2;
paso 12: colocar mueble en enganche_pared;


fin;
```

SMAGORA

Traducción del manual de instrucciones: hay que tener en cuenta que este manual tenía 3 posibles terminaciones, por lo que se han hecho tres archivos txt

```
mueble "SMAGORA";

pieza "panel_lateral"      cantidad 2;
pieza "panel_fondo"        cantidad 1;
pieza "panel_medio"        cantidad 4;

usar herramienta "martillo";
usar herramienta "lapiz";
usar herramienta "destornillador";
usar herramienta "nivelador";

herraje 101345 cantidad 18;
herraje 103693 cantidad 2;
herraje 107716 cantidad 4;
herraje 118331 cantidad 18;
herraje 119252 cantidad 18;
herraje 122628 cantidad 4;
herraje 115753 cantidad 4;
herraje 115754 cantidad 2;
herraje 110646 cantidad 2;

paso 1:  atornillar 118331 cantidad 8 en panel_fondo repetir 2;
paso 2:  atornillar 101345 cantidad 4 en panel_fondo repetir 2;
paso 3: repetir paso 2;
paso 4: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 5: colocar panel_fondo en muebleSemi;
paso 6: colocar panel_lateral en muebleSemi;
```

```
paso 7: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 8: martillar 122628 cantidad 4 en mueble;
paso 9: atornillar 115753 cantidad 2 en panel_fondo usando 107716 cantidad 4;
paso 10: marcar 115753 en pared usando lapiz;
paso 11: atornillar 110646 cantidad 2 en 115753 usando tornillo;
paso 12: atornillar 103693 cantidad 2 en 115753;

fin;
```

Árbol:

2

```
mueble "SMAGORA";

pieza "panel_lateral"       cantidad 2;
pieza "panel_fondo"         cantidad 1;
pieza "panel_medio"         cantidad 4;

usar herramienta "martillo";
usar herramienta "lapiz";
usar herramienta "destornillador";
usar herramienta "nivelador";

herraje 101345 cantidad 18;
herraje 103693 cantidad 2;
herraje 107716 cantidad 4;
herraje 118331 cantidad 18;
herraje 119252 cantidad 18;
herraje 122628 cantidad 4;
herraje 115753 cantidad 4;
herraje 115754 cantidad 2;
herraje 110646 cantidad 2;

paso 1:  atornillar 118331 cantidad 8 en panel_fondo repetir 2;
paso 2:  atornillar 101345 cantidad 4 en panel_fondo repetir 2;
paso 3: repetir paso 2;
paso 4: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 5: colocar panel_fondo en muebleSemi;
paso 6: colocar panel_lateral en muebleSemi;
paso 7: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 8: atornillar 115753 en panel_medio usando 107716 cantidad 2 repetir 2;
paso 9: marcar 115753 en pared usando lapiz;
paso 10: atornillar 110646 cantidad 2 en 115753 usando tornillo;
paso 11: atornillar 103693 cantidad 2 en 115753;

fin;
```

3

```
mueble "SMAGORA";

pieza "panel_lateral"       cantidad 2;
pieza "panel_fondo"         cantidad 1;
pieza "panel_medio"         cantidad 4;
pieza "balda"               cantidad 2;

usar herramienta "martillo";
usar herramienta "lapiz";
usar herramienta "destornillador";

herraje 101345 cantidad 18;
herraje 103693 cantidad 2;
herraje 107716 cantidad 4;
herraje 118331 cantidad 18;
```

```
herraje 119252 cantidad 18;
herraje 122628 cantidad 4;
herraje 115753 cantidad 4;
herraje 115754 cantidad 2;
herraje 110646 cantidad 2;

paso 1:  atornillar 118331 cantidad 8 en panel_fondo repetir 2;
paso 2:  atornillar 101345 cantidad 4 en panel_fondo repetir 2;
paso 3: repetir paso 2;
paso 4: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 5: colocar panel_fondo en muebleSemi;
paso 6: colocar panel_lateral en muebleSemi;
paso 7: atornillar 119252 cantidad 2 en panel_medio repetir 4;
paso 8: martillar 122628 cantidad 4 en mueble;
paso 9: atornillar 118331 cantidad 2 en muebleNuevo;
paso 10: colocar 101345 cantidad 2 en balda;
paso 11: fijar balda en muebleNuevo;
paso 12: atornillar balda usando 119252 cantidad 2;
paso 13: atornillar 115753 cantidad 2 en panel_fondo usando 107716 cantidad 4;
paso 14: colocar smagora en muebleNuevo;
paso 15: marcar 115753 en panel_fondo usando lapiz;
paso 16: deshacer paso 14;
paso 17: atornillar 110646 cantidad 2 en balda usando tornillo;
paso 18: colocar smagora en muebleNuevo;
paso 19: atornillar 103693 cantidad 2 en 115753;

fin;
```

2.2. Nivel avanzado

En el nivel avanzado se nos pedía implementar un analizador para la secuencia de montaje que pueda leer los pasos, escribir la lista de herrajes con sus cantidades por pantalla y la lista de herramientas a utilizar.

2.2.1 Enfoque general

En la gramática definimos, por un lado, la sección de definiciones (definicionHerraje, definicionPieza, usoHerramienta) y, por otro, los pasos de montaje (pasoMontaje, accionLista, accion, herrajeLista, herramientaRef, etc.).
A partir de la entrada obtenemos el árbol de derivación llamando al método programa del parser. Sobre ese árbol aplicamos un visitor específico (AnalizadorMontaje), que recorre las partes relevantes y va actualizando una tabla de símbolos simplificada.
Este enfoque permite separar claramente el reconocimiento sintáctico (realizado por ANTLR) de la lógica de análisis de la secuencia de montaje (implementada en el visitor).

2.2.2. Tabla de símbolos y estructuras de datos

Para materializar la tabla de símbolos hemos empleado dos estructuras, un mapa de herrajes Map<String, Integer> contadorHerrajes = new LinkedHashMap<>(); deonde la clave es el identificador del herraje y el valor la cantidad total usada durante el montaje, y un conjunto de herramientas de la forma Set<String> herramientasUsadas = new LinkedHashSet<>(); que alamcena los nombres de todas las herramientas empleadas en el programa, sin duplicados.
Hemos elegido LinkedHashMap y LinkedHashSet para mantener el orden de inserción, de modo que la salida por pantalla respete el orden en que los elementos se van encontrando en el texto fuente.

2.2.3. Recorrido del árbol y actualización de la tabla

El Visitor creado extiende el base y redefine los nodos que aportan información útil solamente, que son herrajes y herramientas utilizados. Para el primero se ha reescrito el método visitHerrajeItem correspondiente a la regla.

```
72    herrajeItem      : herrajeRef (CANTIDAD NUM)? ;
```

Dentro de el método se obtiene un identificador del herrajeRef, si es un nombre se normaliza eliminando las comillas en caso de ser una CADENA. Y si es un NUM, se usa el propio número como identificador. Se calcula a su vez la cantidad asociada, siendo el por defecto 1, y si aparece el patrón CANTIDAD NUM, se toma el valor numérico indicado. Finalmente se actualiza el mapa mediante merge. De esta manera el contadorHerrajes contiene al finalizar el recorrido el uso acumulado de cada tipo de herraje en todos los pasos.
Para el segundo debemos tener en cuenta que aparecen en dos contextos diferentes:

```
24  usoHerramienta
25      : USAR HERRAMIENTA CADENA PUNTOYCOMA
26      ;
```

```
75    herramientaRef  : nombre ;
```

En el método visitUsoHerramienta se extrae el contenido de la CADENA, se eliminan las comillas y se registra la herramienta en el conjunto de herramientasUsadas.

En el método visitHerramientaRef se llama a una función auxiliar que extrae el nombre y se vuelve a registrar en el conjunto; set fuerza la nula aparición de duplicados.

Para el resto de los nodos hemos optado por recurrir a la función visitChildren(ctx) de modo que el árbol se recorre completo pero solo realiza trabajo adicional en los puntos donde realmente se actualiza la tabla de símbolos.

2.2.4. Salida por pantalla

Una vez terminado el recorrido en la clase AnalizadorMontaje tenemos un método imprimirResumen() que muestra la información que se nos pedía:

- Lista de herrajes con sus cantidades: se recorre el Map<String,Integer> y se imprime cada par (herraje, cantidad) en una línea.
- Lista de herramientas a utilizar: se recorre el conjunto de herramientas y se imprime cada una en una línea independiente.

Este método se invoca desde una clase auxiliar MainMontaje responsable de leer el fichero de entrada, ejecutar el lexer y parser generados por antlr, llamar al visitor sobre el programa y presentar el resultado.

```
C:\antlr\muebles_gramatica>java MainMontaje libreria_kallax1.txt
==== RESUMEN DEL MONTAJE ====

>> Herrajes utilizados (acumulados en los pasos):
   - 104321: 8
   - 121030: 6
   - 100092: 2
   - 101339: 14
   - 115753: 3
   - 100372: 2
   - tornillo: 2
   - 103693: 2

>> Herramientas utilizadas:
   - destronillador punta plana
   - destronillador estrella
   - lapiz
   - martillo
   - nivel
```

Ilustración 1Ejemplo con Mueble Kallx Manual 1

2.2.5. Ampliaciones como consecuencia de decisiones de diseño

Debido a las decisiones de diseño tomadas, tenemos que manejar la construcción repetir NUM para poder expresar de manera correcta el número de herrajes correctos usados. Esto se ha llevado a cabo teniendo en cuenta un factor de repetición por el cual se multiplica las cantidades usadas de dicho herraje.

3. Anexo 1

A la hora de adjuntar los árboles de los distintos programas de las librerías de muebles de IKEA, se veían muy mal ya que nos salían en horizontal y no eran legibles, por ello adjuntamos una carpeta con todas las fotos de dichos arboles con el nombre de cada uno.